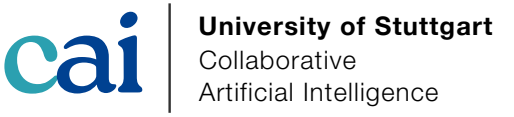


Machine Perception and Learning

for Collaborative Intelligent Systems

Prof. Dr. Andreas Bulling

WS 2025/2026

Collaborative Artificial Intelligence Group, University of Stuttgart

www.collaborative-ai.org

The Goal of this Lecture

From nothing …

The Goal of this Lecture

From nothing ...

... to one of the most ubiquitously used Reinforcement Learning algorithms: PPO.

Reinforcement Learning

- Introduction
- Markov Decision Process
- Temporal-difference Learning
- Deep Reinforcement Learning

Reinforcement Learning

Introduction

Slide credits for today:

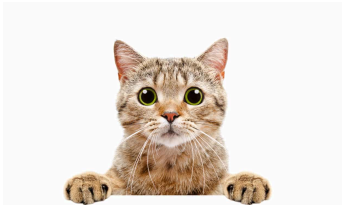
- Schulman [2017]
- O. Hilliges @ETHZ
- Multi-Agent Reinforcement Learning: Foundations and Modern Approaches by Stefano V. Albrecht, Filippos Christianos and Lukas Schäfer

- Supervised Learning
  - Learn a mapping from input to output
  - Ground truth data available (e.g. class labels)

- Supervised Learning
  - Learn a mapping from input to output
  - Ground truth data available (e.g. class labels)
- Unsupervised Learning
  - Learn the structure of a dataset
  - No ground truth data

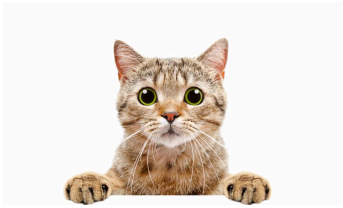
- Supervised Learning
  - Learn a mapping from input to output
  - Ground truth data available (e.g. class labels)
- Unsupervised Learning
  - Learn the structure of a dataset
  - No ground truth data
- Reinforcement Learning
  - Data revealed through interacting with the environment
  - Learn how to act

- Supervised Learning: This is a cat



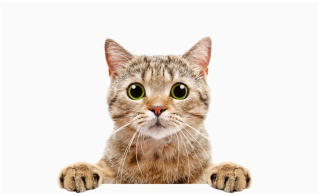
Source: Sonsedska/Getty Images/iStockphoto

- Supervised Learning: This is a cat



Source: Sonsedska/Getty Images/iStockphoto

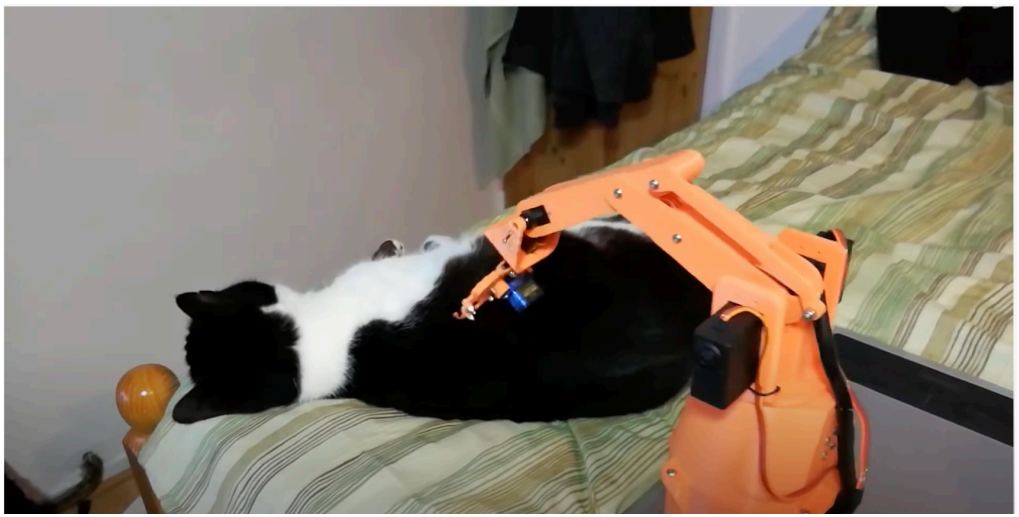
- Unsupervised Learning: These two things are similar



Source: Sonsedska/Getty Images/iStockphoto

Source: Unsplash @theluckyneko

- Reinforcement Learning: This thing likes to be scratched

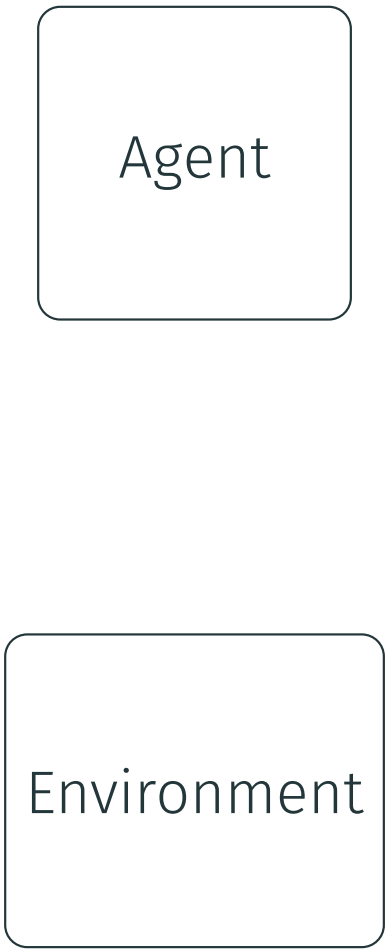


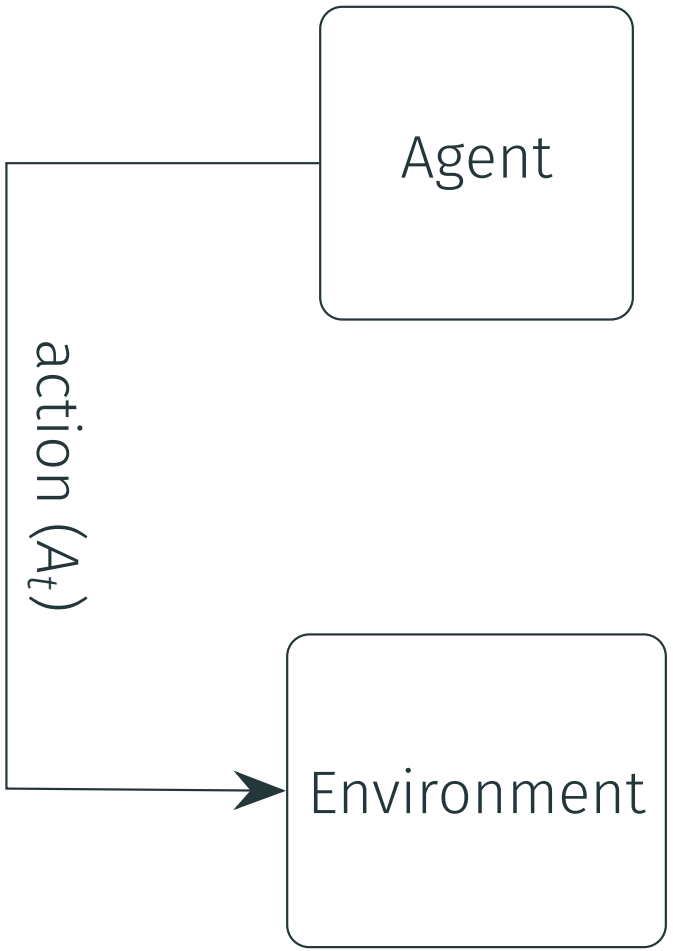
Source: Youtube @isaac879

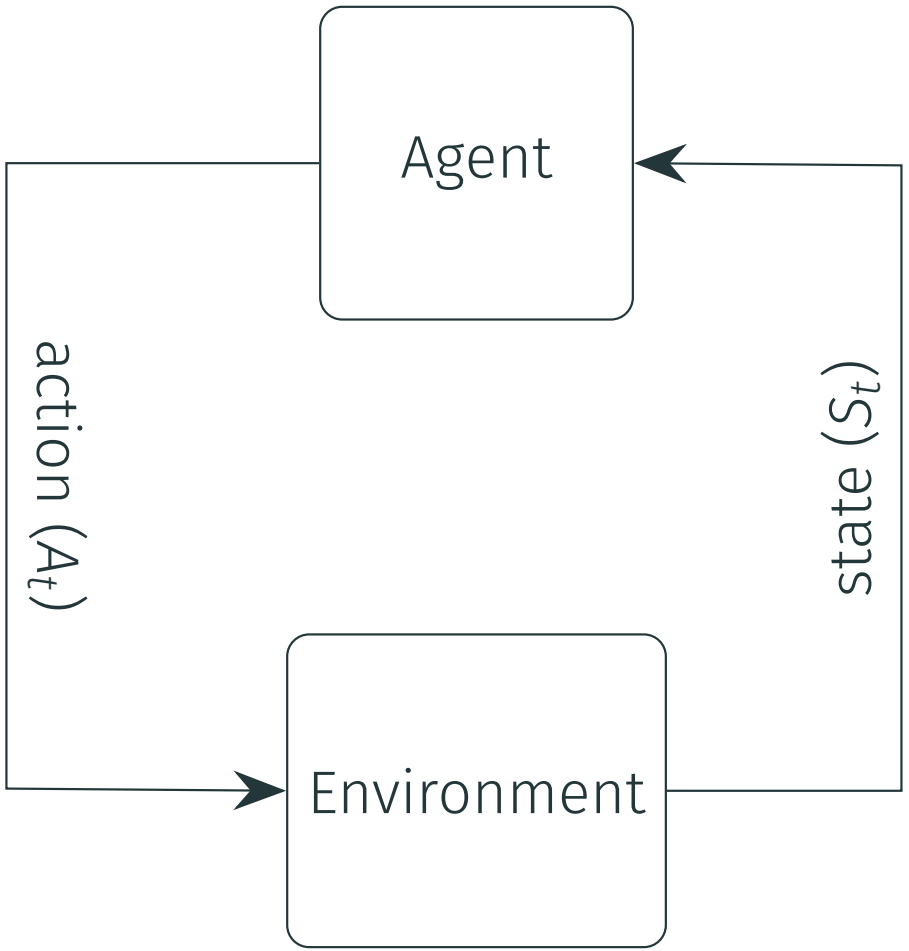
Reinforcement learning (RL)

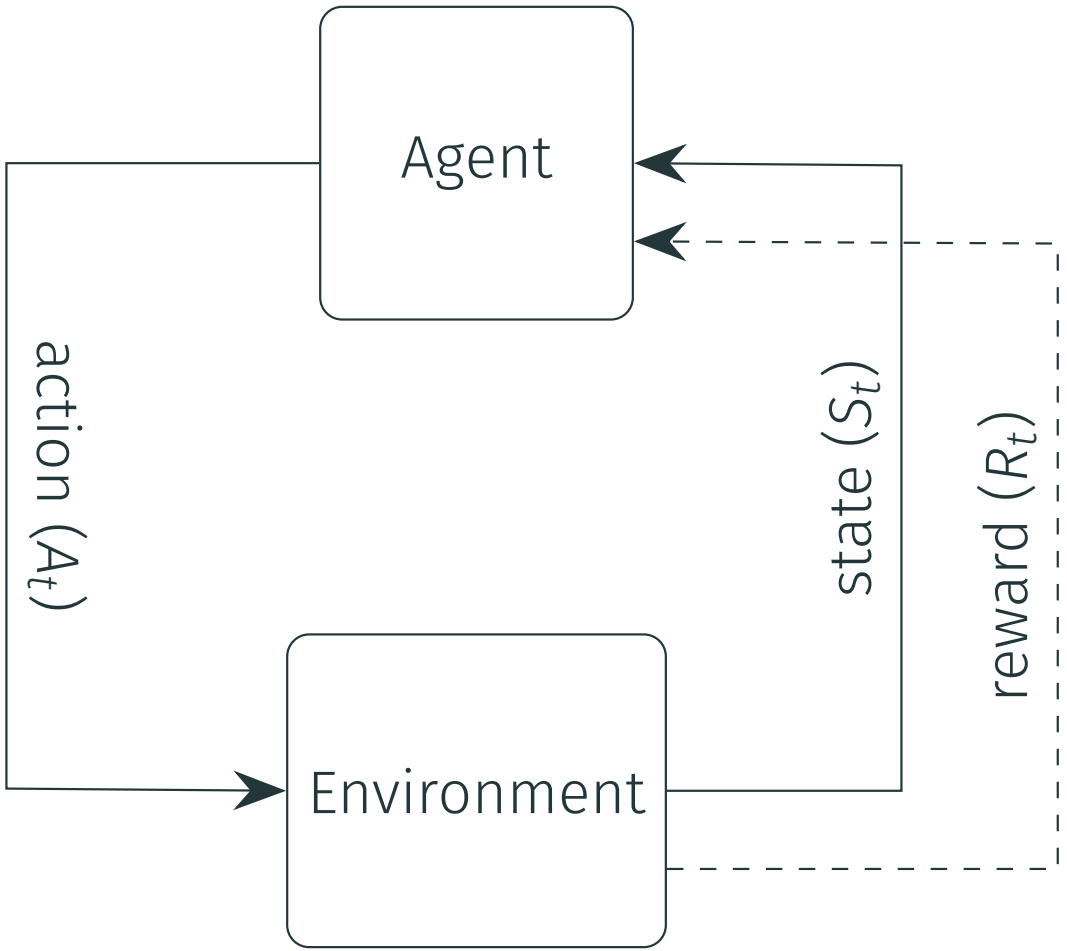
Learning to solve sequential decision problems via *repeated interaction with environment* (trial and error)

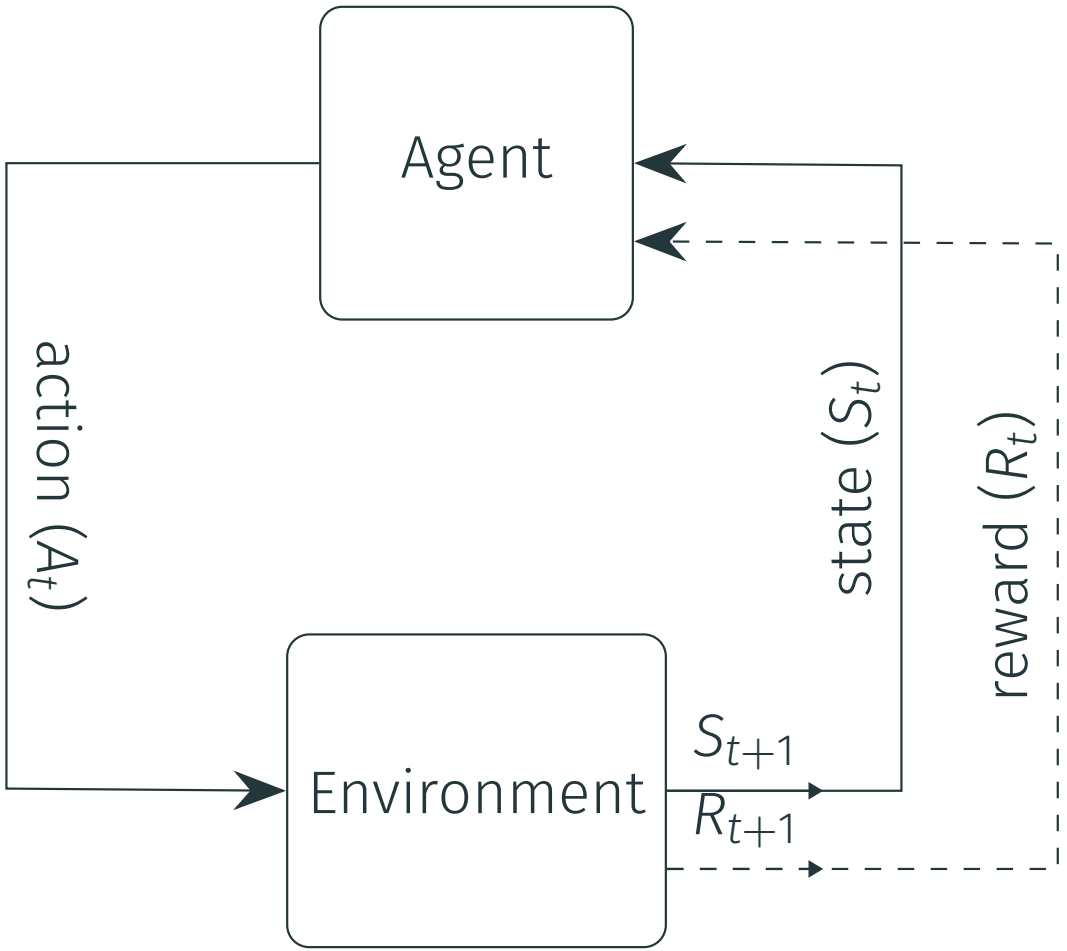
1. What is a sequential decision problem?
2. What does it mean to “solve” the problem?
3. What is learning by interaction?


Agent
Environment

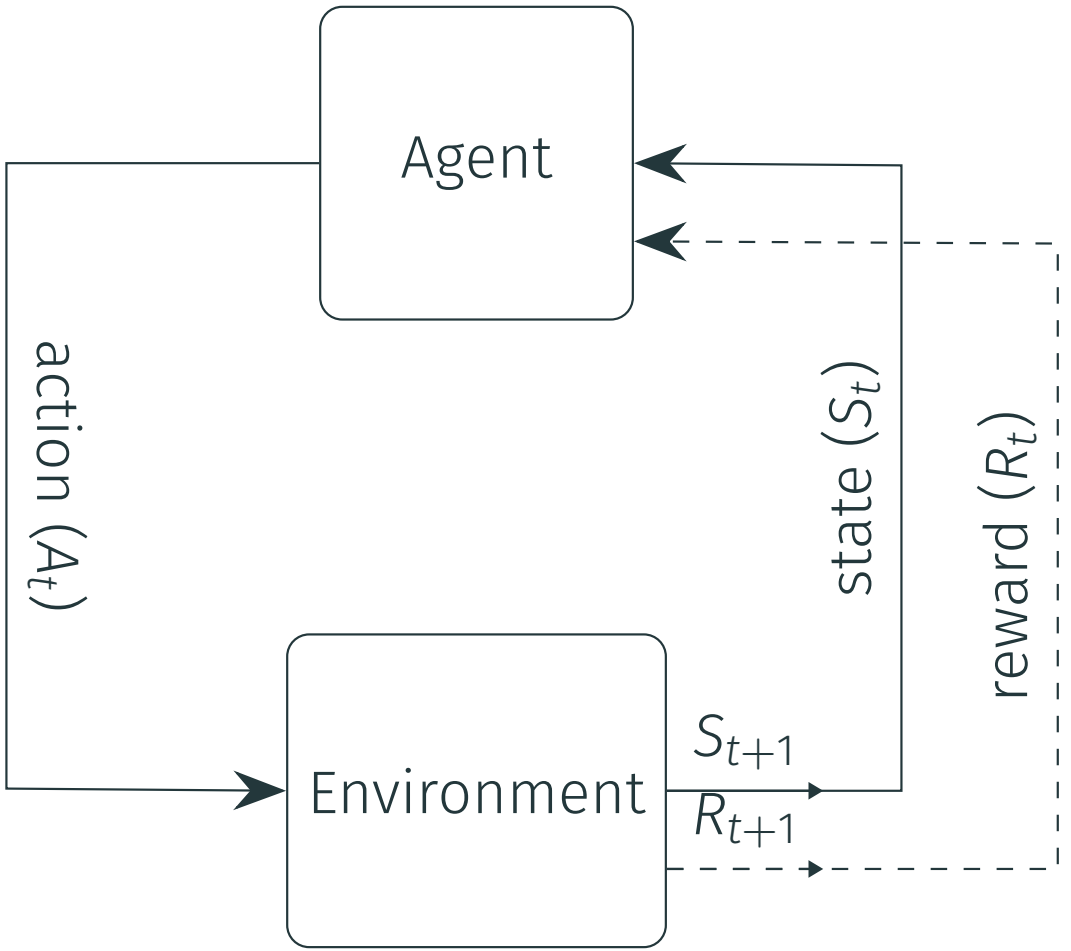

Agent
action (A_t)
Environment

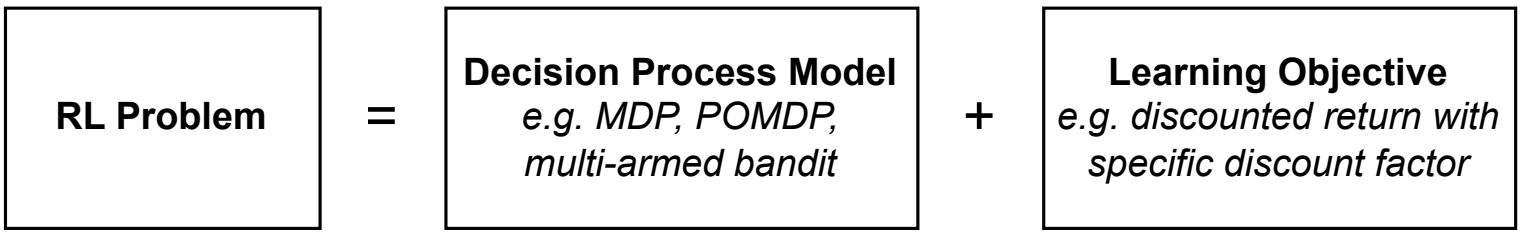

Agent
action (A_t)
state (S_t)
Environment


Agent
Environment
action (A_t)
state (S_t)
reward (R_t)


Agent
action (A_t)
state (S_t)
reward (R_t)
Environment
S_{t+1}
R_{t+1}

Policy $\pi(a|s)$



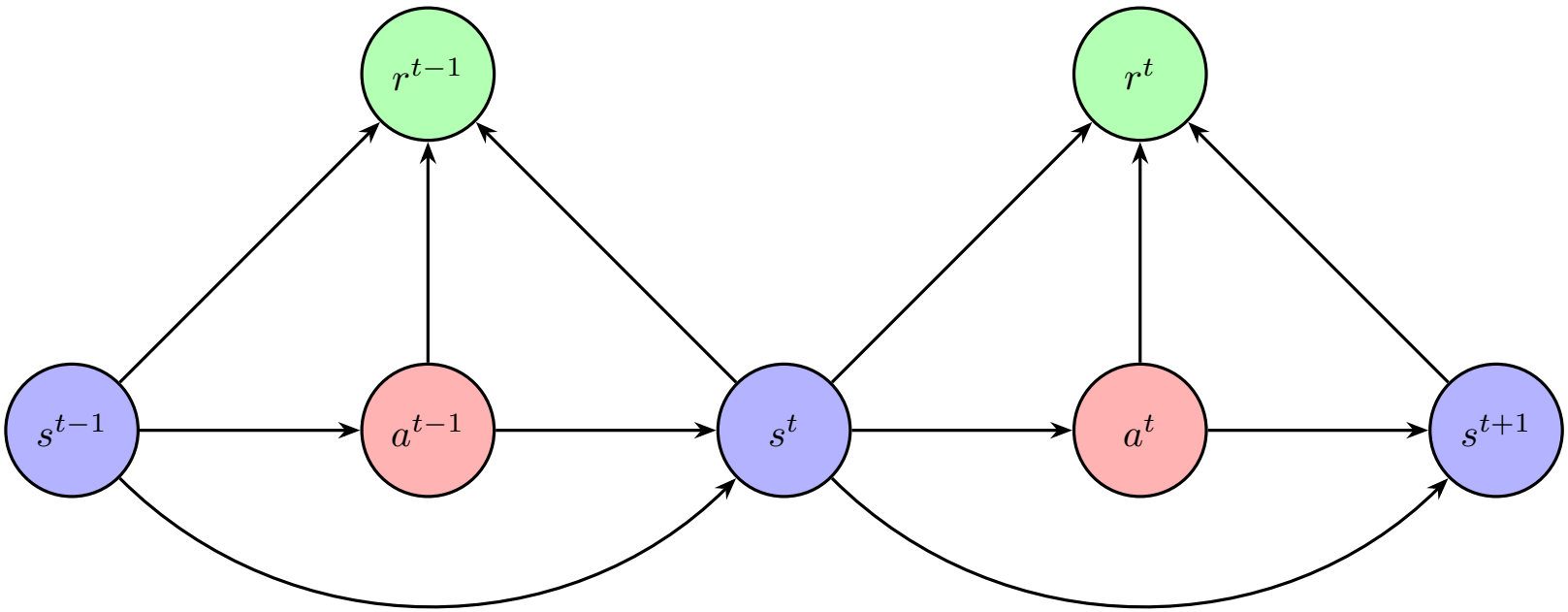


Source: Albrecht et al. [2024]

1. Sequential decision process is defined formally as Markov decision process (MDP)
2. Goal is to maximise total rewards received → Learning: find best actions by *trying* them

Reinforcement Learning

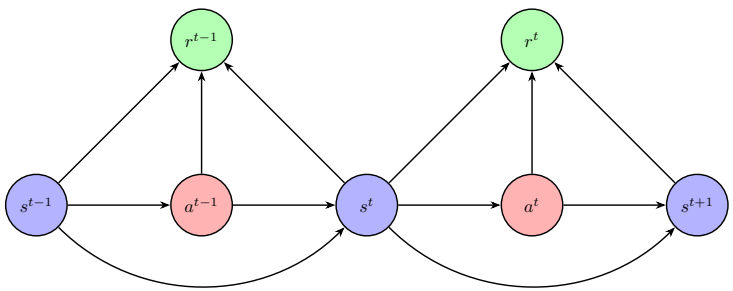
Markov Decision Process



Source: Albrecht et al. [2024]

MDP is defined as:

1. S: Finite set of states, with subset of terminal states $\bar{S} \subset S$
2. A: Finite set of actions
3. $\mathcal{R}$: Reward function
   $\mathcal{R} : S \times A \times S \to \mathbb{R}$
4. $\mathcal{T}$: State transition function
   $\mathcal{T} : S \times A \times S \to [0, 1]$
5. μ: Initial state distribution
   $\mu : S \to [0, 1]$



Source: Albrecht et al. [2024]

Using RL, learn policy π that maximizes the expected **return**:

- Returns are the sum of rewards received over time
- If MDP is non-terminating (i.e. $t \to \infty$), we **discount** returns to ensure finite (discounted) returns

$$\mathbb{E}_\pi[G_t] = \mathbb{E}_\pi[r_{t+1} + \gamma r_{t+2} + \ldots] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}\right]$$

- $\gamma \in [0, 1]$, $\gamma \to 0$ = "myopic", $\gamma \to 1$ = "far-sighted"
- π is the policy that determines which actions are chosen

- State-value function $V_\pi(s)$ of a MDP is the expected return (cumulative reward) starting from state s, following the policy
- $V_\pi : S \to \mathbb{R}$ for policy π

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

- The **optimal** state-value function $V^*(s)$ is the maximum value function over all policies

$$V^*(s) = \max_\pi V^\pi(s)$$

- Action-value function $Q^\pi(s, a)$ – expected return starting from state s, taking action a, and following policy

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

(more on this later)

- The **optimal** action-value function $Q^*(s, a)$ is the maximum action-value function over all policies

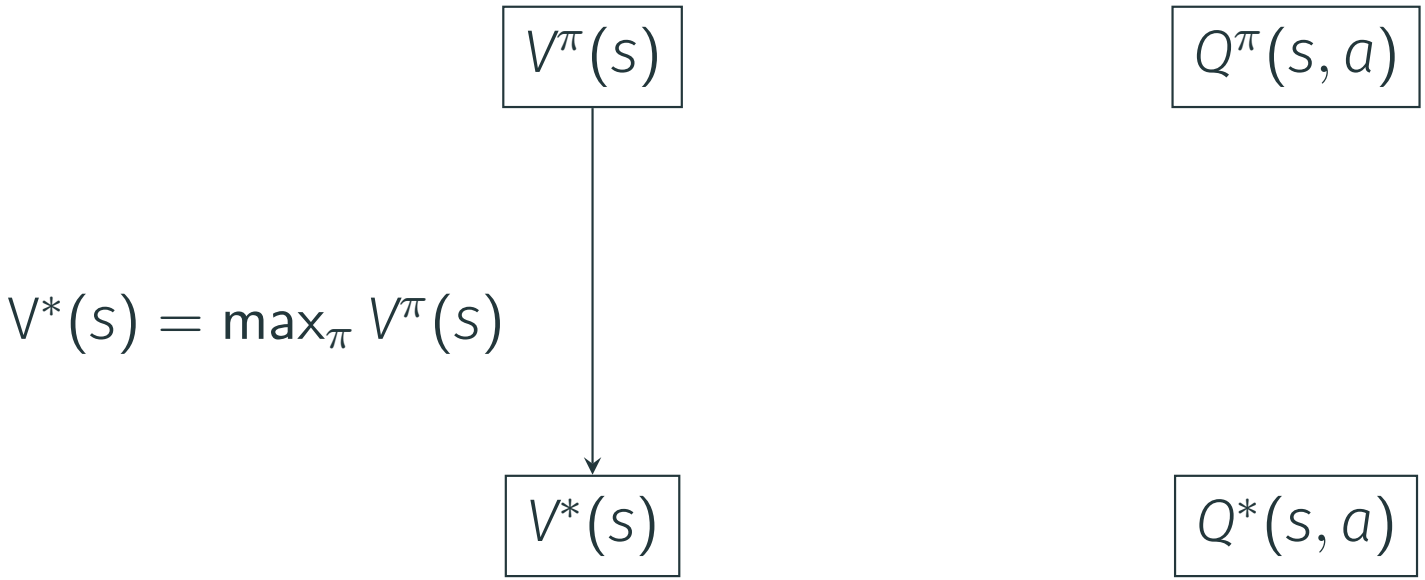
$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

(more on this later)

- Value functions measure the goodness of a particular state or state/action pair
  - How good is it for an agent to be in a particular state or execute a particular action at a particular state for a given policy
- Optimal value functions measure the best possible states or state/action pairs under all possible policies

State value functions

Action value functions



State value functions

$V^{\pi}(s)$

$V^{*}(s) = \max_{\pi} V^{\pi}(s)$

$V^{*}(s)$

Action value functions

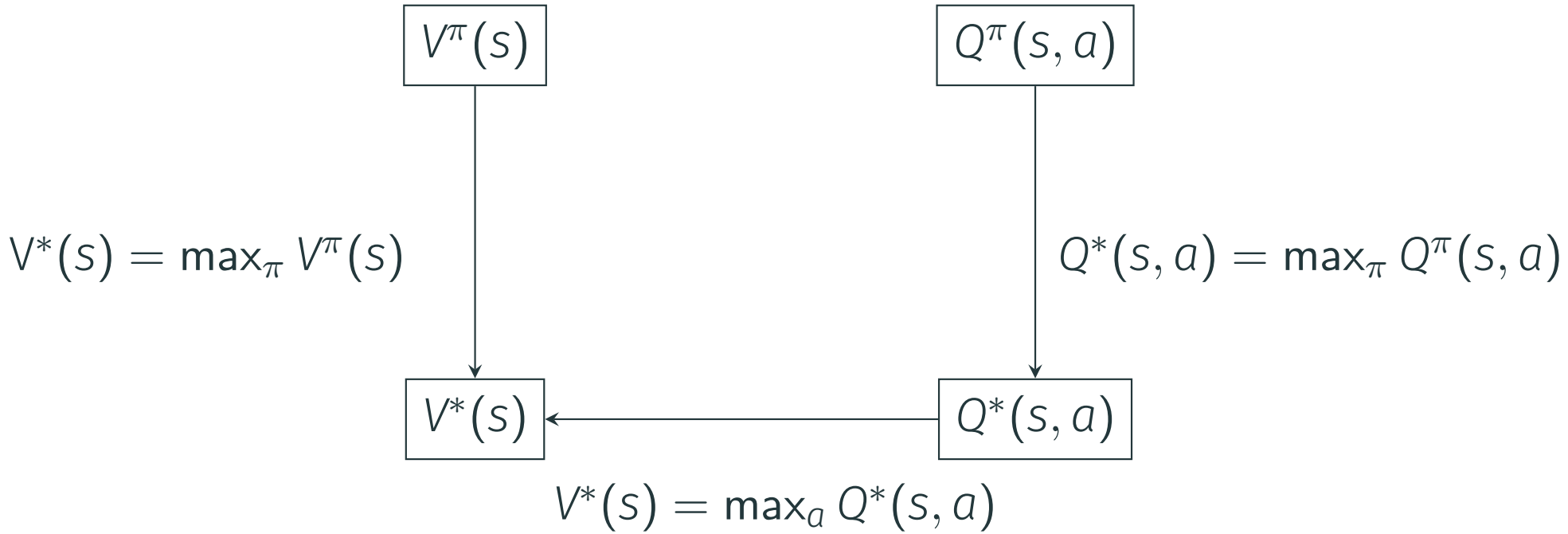
$Q^{\pi}(s, a)$

$Q^{*}(s, a) = \max_{\pi} Q^{\pi}(s, a)$

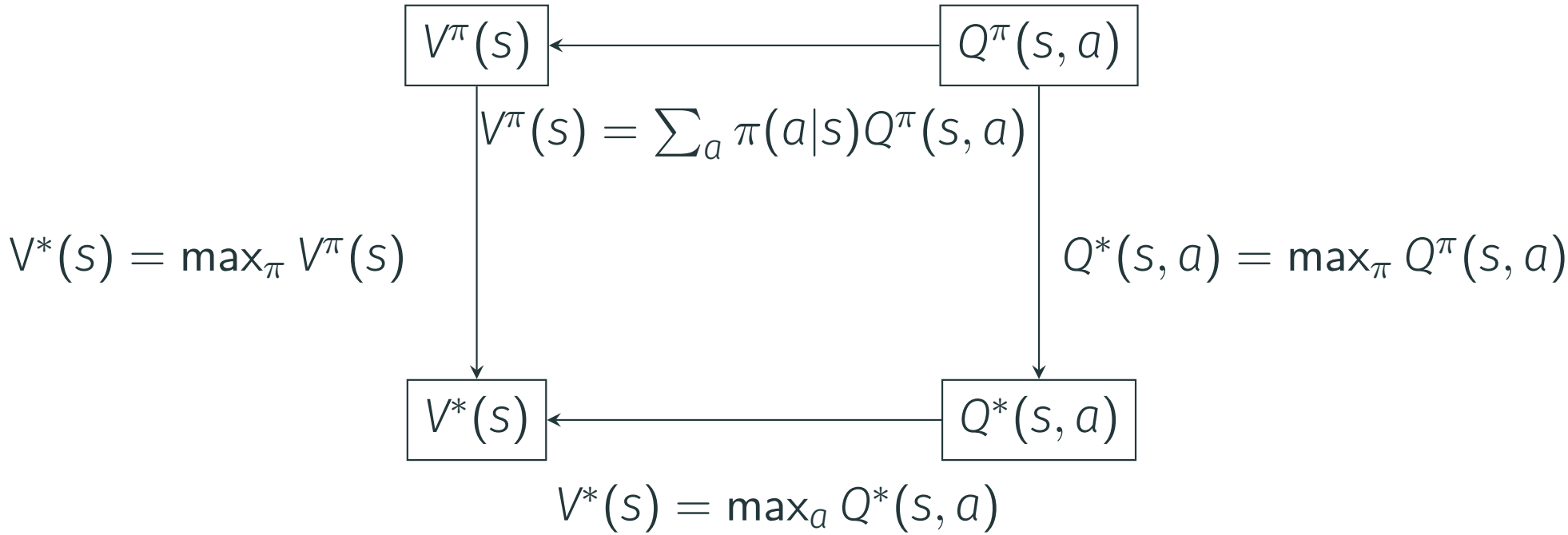
$Q^{*}(s, a)$

State value functions

Action value functions



State value functions Action value functions



- The optimal policy can be found by maximising over $Q^*(s, a)$

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \text{argmax}_a Q^*(s, a) \\ 0, & \text{else} \end{cases} \tag{1}$$

- The optimal policy can also be found by maximising over $V^*(s')$ with a one-step look ahead

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \text{argmax}_a[\sum_{s'} p(s'|s,a)(r(s,a,s') + \gamma V^*(s'))] \\ 0, & \text{else} \end{cases} \tag{2}$$

How do we find $Q^*(s, a)$ and $V^*(s)$?
Recursively:

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} \dots \\ &= r_{t+1} + \gamma (r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

How do we find $Q^*(s, a)$ and $V^*(s)$?
By taking expectations:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s] \end{aligned}$$

How do we find $Q^*(s, a)$ and $V^*(s)$?
By taking expectations:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a|s) \mathbb{E}_{s'}[r(s, a, s') + \gamma V^\pi(s')] \end{aligned}$$

How do we find $Q^*(s,a)$ and $V^*(s)$?
By taking expectations:

$$
\begin{aligned}
V^\pi(s) &= \mathbb{E}_\pi[G_t|S_t = s] \\
&= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1}|S_t = s] \\
&= \mathbb{E}_\pi[r_{t+1} + \gamma V^\pi(S_{t+1})|S_t = s] \\
&= \sum_a \pi(a|s)\mathbb{E}_{s'}[r(s,a,s') + \gamma V^\pi(s')] \\
&= \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)[r(s,a,s') + \gamma V^\pi(s')]
\end{aligned}
$$

How do we find $Q^*(s,a)$ and $V^*(s)$?
By taking expectations:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t|S_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1}|S_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma V^\pi(S_{t+1})|S_t = s] \\ &= \sum_a \pi(a|s)\mathbb{E}_{s'}[r(s,a,s') + \gamma V^\pi(s')] \\ &= \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)[r(s,a,s') + \gamma V^\pi(s')] \end{aligned}$$

Similarly for

$$Q^\pi(s,a) = \sum_{s'} p(s'|s,a)\left((r(s,a,s') + \gamma \sum_{a'} \pi(a'|s')Q^\pi(s',a')\right)$$

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} p(s'|s,a)[r(s,a,s') + \gamma V^{\pi}(s')]$$

Solve the linear system:

- variables: $V^{\pi}(s)$ for all s
- constants: $p(s'|s,a), r(s,a,s')$

Solve by iterative methods:

$$V^{\pi}_{[k+1]}(s) = \sum_{a} \pi(a|s) \sum_{s'} p(s'|s,a)[r(s,a,s') + \gamma V^{\pi}_{[k]}(s')]$$

- Dynamic Programming
  - Requires MDP with given environment model (transition probabilities)
  - Value function based on estimates of subsequent values
- Monte-Carlo Methods (not covered here)
- Temporal-Difference Learning
  - No need to know environment model
  - Value function updated based on estimates of subsequent values along a trajectory

- Dynamic Programming (DP) is a family of algorithms to compute **optimal value functions** and **optimal policies in MDPs**.
- In DP, we **assume complete knowledge** of the MDP, including the transition and reward function $(\mathcal{T}, \mathcal{R})$.
- Given complete knowledge, we can use the **Bellman equation** to find optimal value functions and policies.
- Typically impractical but high theoretical utility

Policy Iteration is a DP algorithm that alternates between:

- **Policy evaluation**: compute value function V^π for current policy π
- **Policy improvement**: improve current policy π with respect to V^π

$$\pi^0 \rightarrow V^{\pi^0} \rightarrow \pi^1 \rightarrow V^{\pi^1} \rightarrow \pi^2 \rightarrow \ldots \rightarrow V^* \rightarrow \pi^*$$

Value Iteration uses the **Bellman optimality equation** as update operator:

$$V^{k+1}(s) \leftarrow \max_{a \in A} \sum_{s' \in S} p(s'|s,a)[r(s,a,s') + \gamma V^k(s')], \quad \forall s \in S$$

The equation expresses the value of a state as the maximum expected return achievable by taking the best action and then following the optimal policy.

Algorithm 1 Value Iteration

Initialize: $V(s) = 0, \forall s \in S$

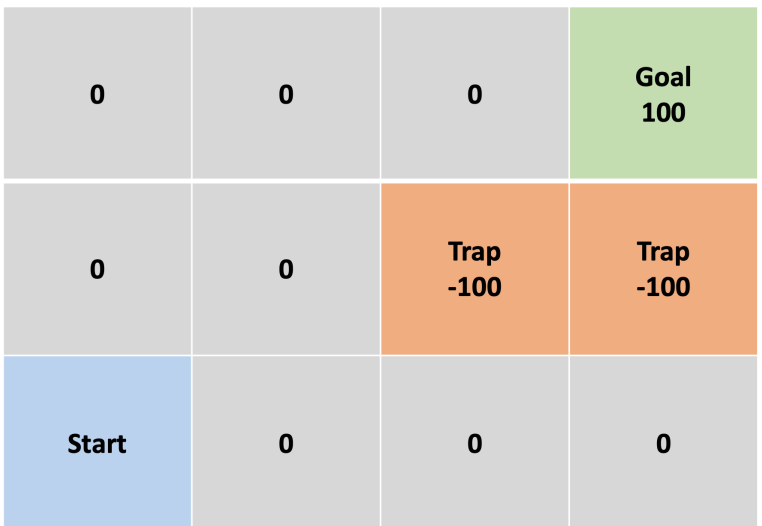
repeat

$\forall s \in S : V(s) \leftarrow \max_{a \in A} \sum_{s' \in S} p(s'|s,a) \left[r(s,a,s') + \gamma V(s') \right]$

until V converged **return** optimal policy π^* with:

$\forall s \in S : \arg\max_{a \in A} \sum_{s' \in S} p(s'|s,a) \left[r(s,a,s') + \gamma V(s') \right]$

Goal: Find optimal path to goal



- States: Grid fields, start state blue field, terminal states trap/goal
- Actions: Move left/right/up/down
- Reward: 100 upon reaching the goal, -100 landing in the trap, -5 for each step taken
- Other info: $\gamma = 1$, initial states initialized to 0, deterministic transitions

Initial value is 0 for every state

0	0	0	Goal 100
0	0	Trap -100	Trap -100
Start 0	0	0	0

Iteration 1

-5	-5	95	Goal 100
-5	-5	Trap -100	Trap -100
Start -5	-5	-5	-5

Iteration 2

-10	90	95	Goal 100
-10	-10	Trap -100	Trap -100
Start -10	-10	-10	-10

Iteration 3

85	90	95	Goal 100
-15	85	Trap -100	Trap -100
Start -15	-15	-15	-15

Iteration 4

85	90	95	Goal 100
80	85	Trap -100	Trap -100
Start -20	80	-20	-20

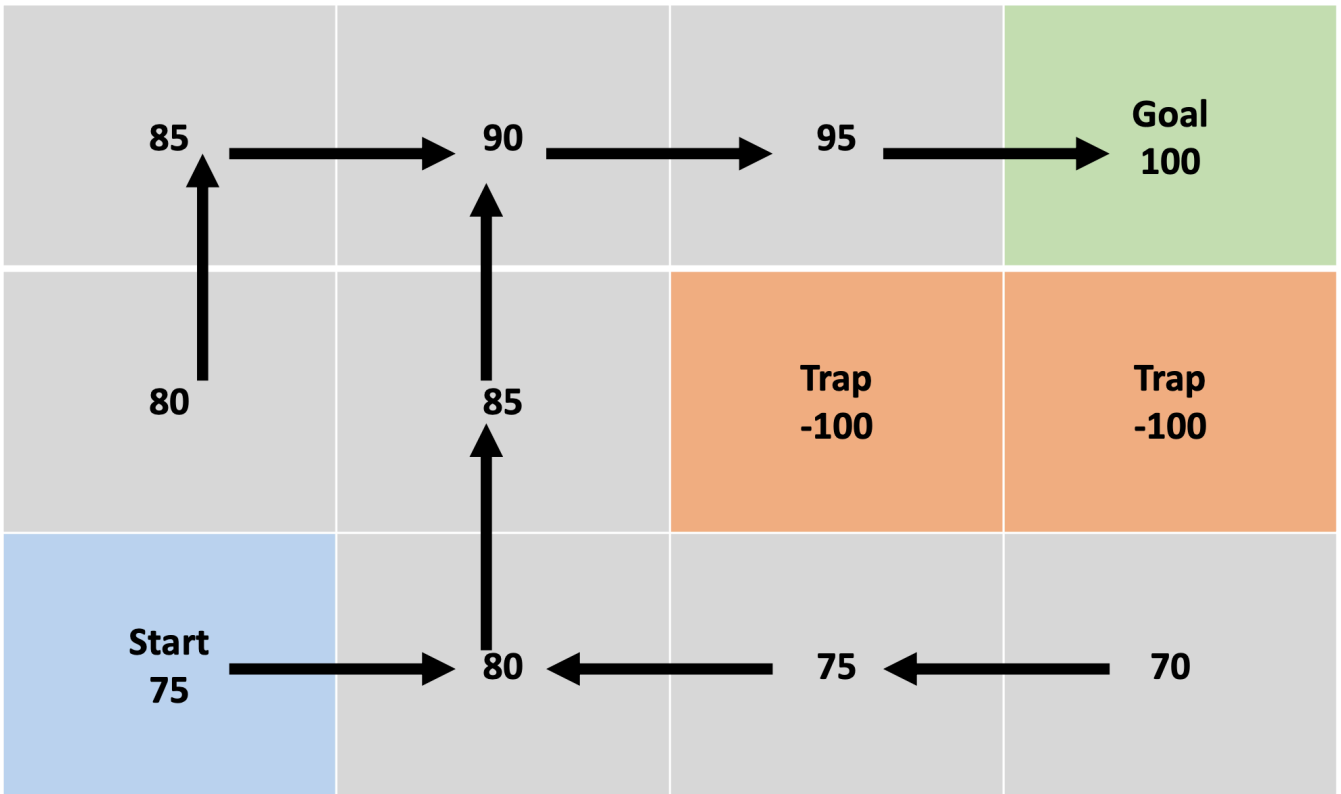
Iteration 5

85	90	95	Goal 100
80	85	Trap -100	Trap -100
Start 75	80	75	-25

Iteration 6. Done as there is no change anymore.

85	90	95	Goal 100
80	85	Trap -100	Trap -100
Start 75	80	75	70

Compute greedy policy



Pros

- Exact methods
- Policy/Value iterations are guaranteed to converge in a finite number of iterations
- Value iteration is typically more efficient

Cons

- Need to know the transition probability matrix
- Need to iterate over the whole state space (very expensive)
- Requires memory proportional to the size of the state space

- Assume that the state space is too big to iterate
- Given a policy π: How can we estimate the value of those states?

Reinforcement Learning

Temporal-difference Learning

- We use π to visit states
- We don't update the whole state space, but only visited states
- For each step with action a from s to s' that we take with our policy, we compute the difference to our current estimate and update our value function (known as **bootstrapping**):

$$\Delta V(s) = \underbrace{r(s,a) + \gamma V(s')}_{\text{Real (Observation)}} - \underbrace{V(s)}_{\text{Estimation}}$$

$$V(s) \leftarrow V(s) + \alpha \Delta V(s)$$

where $\alpha > 0$ is the learning rate

What policy can we use to interact with our environment?

- **Random policy** (Exploration): In each state, choose an action randomly.
  - Visit states close to starting point more often
  - Faraway states get neglected
- **Greedy policy** (Exploitation): In each state, always choose the best action
  - Can find reward quickly
  - Can get stuck in a local minimum

We need to find a balance between:

exploration selecting random actions to visit unknown states

exploitation selecting actions that are optimal given our current knowledge

What could be a good trade-off? ϵ-greedy policy

- In each state, with small probability ϵ choose randomly, else choose greedily
- Works well in practice but more sophisticated approaches exist

Temporal-difference Learning – Two major implementations

- SARSA (State–action–reward–state–action)
  - On-Policy: Computes the Q-Value according to a policy and the agent follows that policy.
- Q-Learning
  - Off-Policy: Computes the Q-Value according to a greedy policy but the agent follows a different exploration policy.

For each step with action a from s to s' that we take with our policy (also a' under s'), we compute the difference to our current estimate and update our value function like:

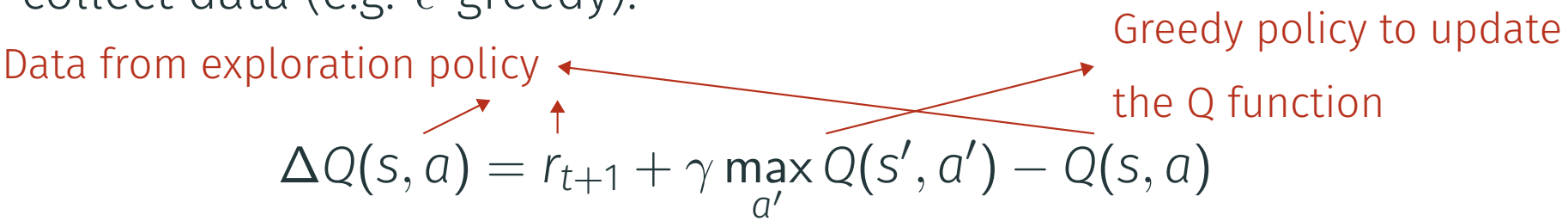
$$\Delta Q(s,a) = r_{t+1} + \gamma Q(s',a') - Q(s,a)$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha \Delta Q(s,a),$$

$$\alpha > 0 \text{ (learning rate)}$$

Sarsa is an on-policy algorithm, since we use the same policy for data collection and function update.

Q-learning is an off-policy algorithm because the policy we use to update the Q function is different from the policy we use to collect data (e.g. ϵ-greedy).



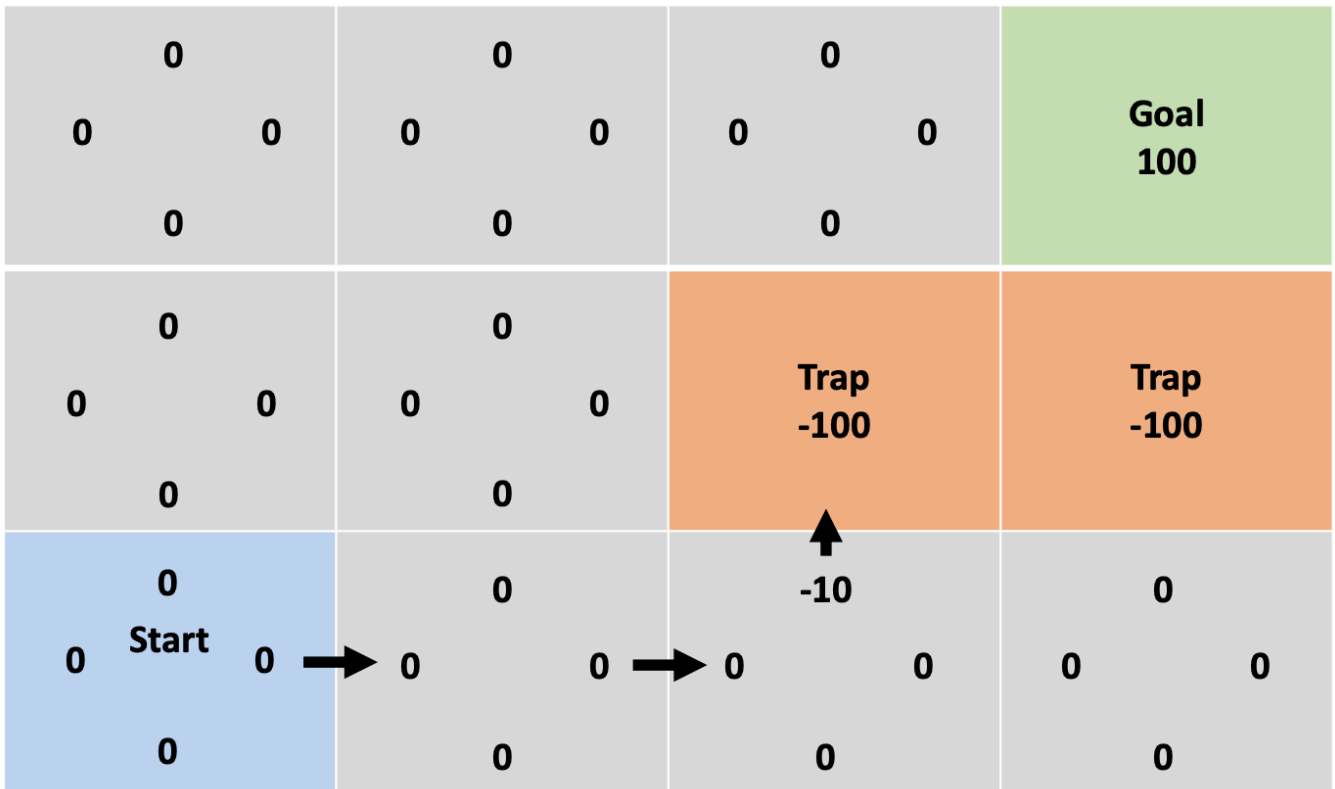
$$Q(s,a) \leftarrow Q(s,a) + \alpha \Delta Q(s,a),$$

$$\alpha > 0 \text{ (learning rate)}$$

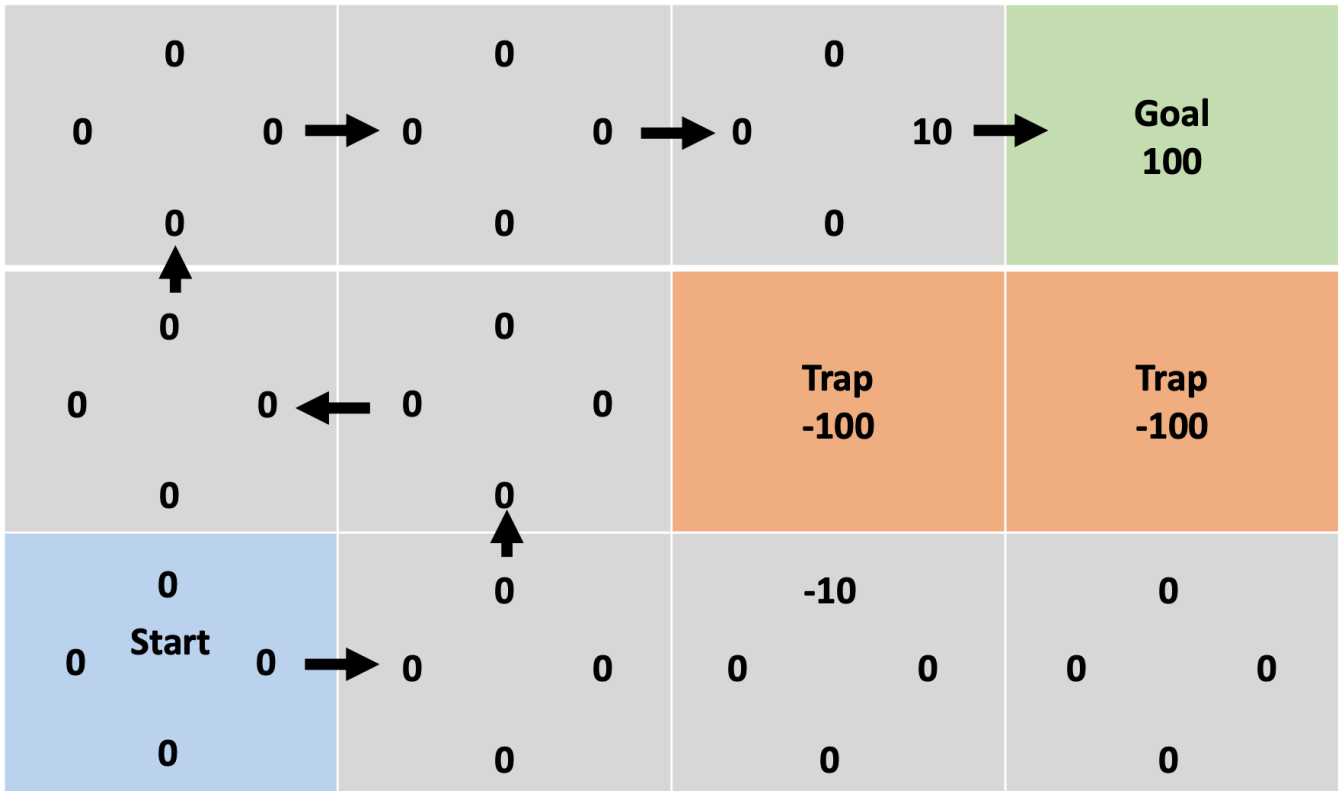
Initial Q function

0 0 0 0	0 0 0 0	0 0 0 0	Goal 100
0 0 0 0	0 0 0 0	Trap -100	Trap -100
0 0 Start 0 0	0 0 0 0	0 0 0 0	0 0 0 0

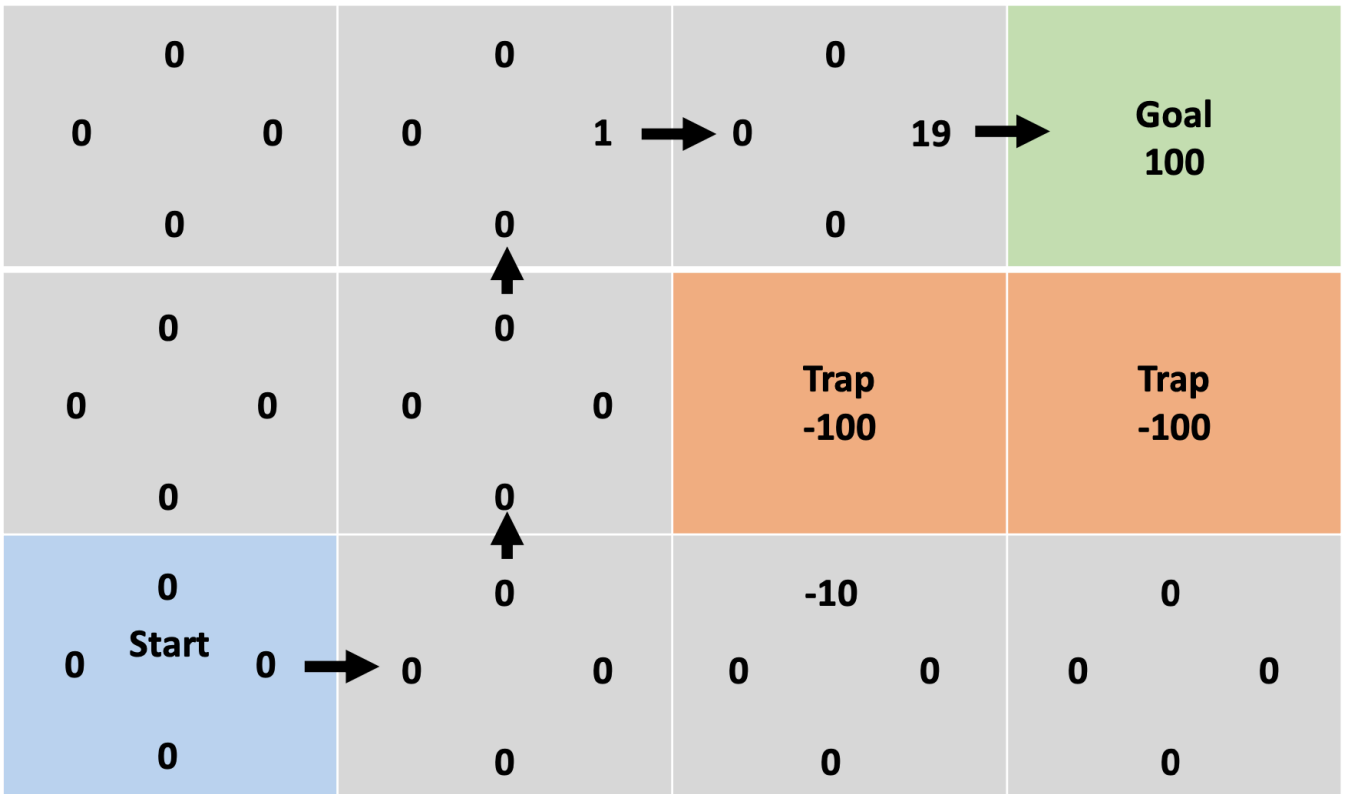
Episode 1: Step-by-step updates of Q-values



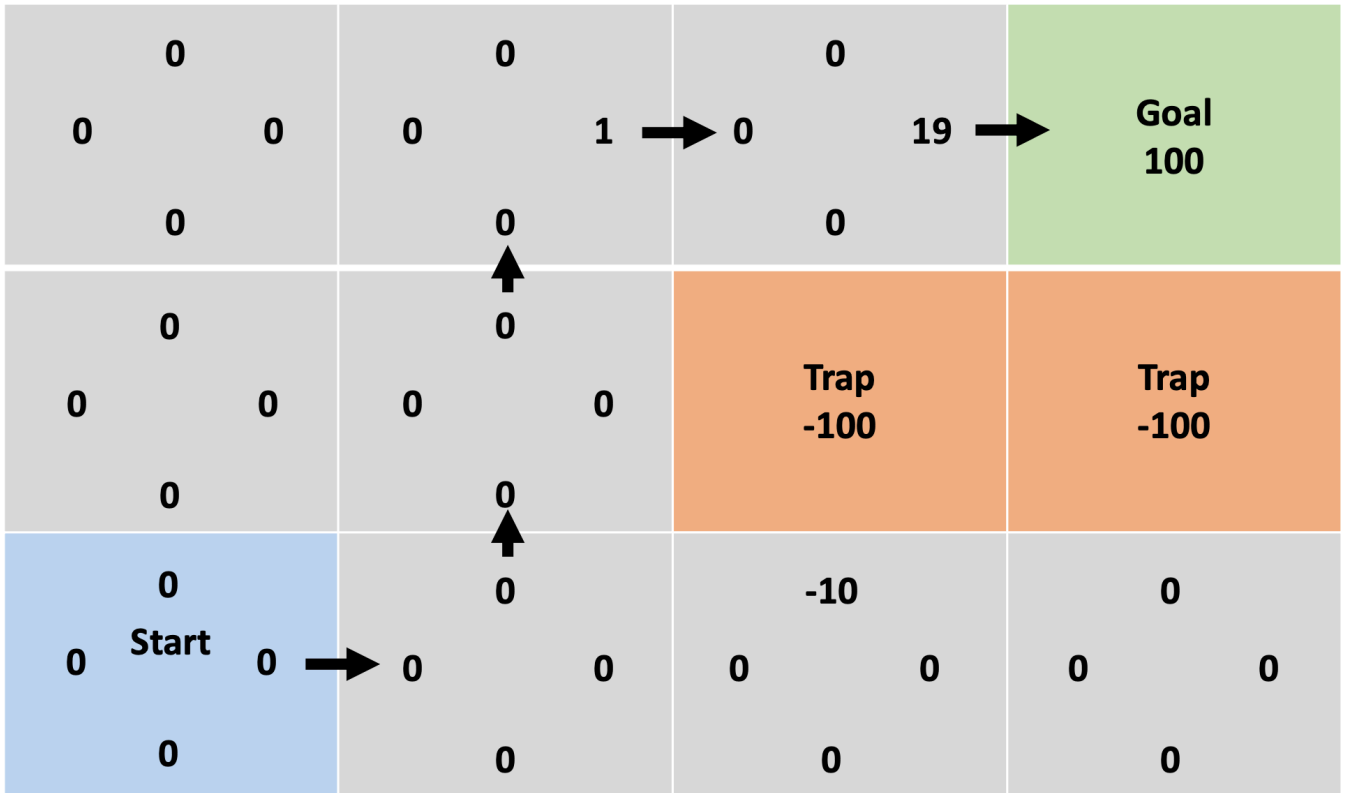
Episode 2: Step-by-step updates of Q-values



Episode 3: Step-by-step updates of Q-values



Keep running until convergence



Pros

- Less variance due to bootstrapping
- More sample efficient
- Do not need to know the transition probability matrix

Cons

- Biased due to bootstrapping
- Exploration vs. Exploitation Dilemma
- Can behave poorly in stochastic environments

Reinforcement Learning

Deep Reinforcement Learning

Tabular Learning

- We learn about each state separately
- No generalisation
- For image-input and larger state spaces, Q-learning is most likely to fail

We need a concept that works for such cases

- Needs to approximate values of states that have not been visited

Policy $\pi : S_t \rightarrow A_t$

Value $v : S_t \rightarrow V(S_t)$

These are nothing else than **functions**, we need a way to represent and learn them.

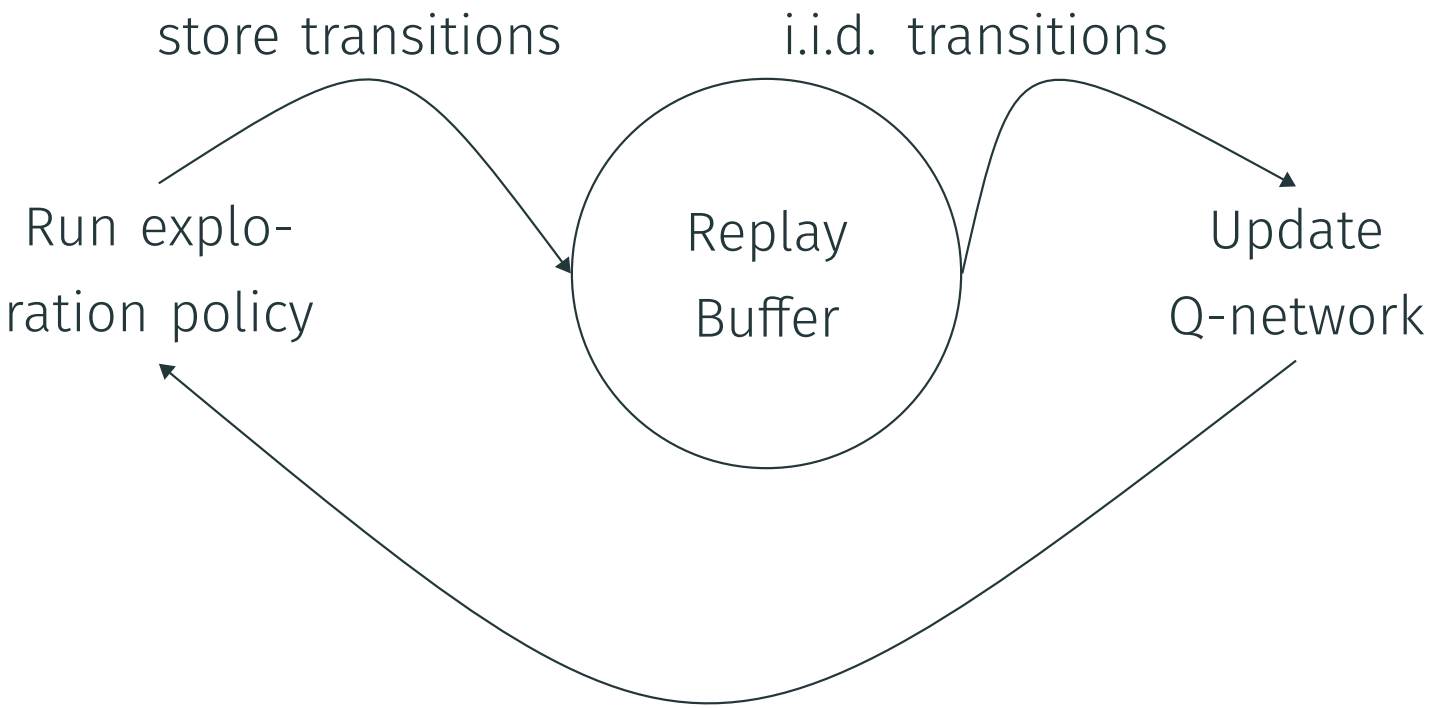
$\rightarrow$ Use function approximation to learn the value function!

$\rightarrow$ E.g. Deep Learning!

- Previously, we assumed a separate Q-value estimate for each state-action pair (s, a).
- The Q-learning updates reduce to **stochastic gradient descent** on:

$$Loss(\theta) = (y - Q_\theta(s, a))^2, \quad y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$$

- SGD assumes that the updates we apply are **i.i.d.** (independent and identically distributed)
- In RL, values of states visited in a trajectory are strongly correlated
- How can we address this?
  - Use a **replay buffer** to store generated samples
  - During updates, randomly sample transitions from the buffer
  - Since Q-learning is off-policy, we are allowed to use old samples



$$(R + \gamma \max_{a'}\{Q_\theta(s', a')\} - Q_\theta(s, a))^2$$

- Observations are high-dimensional and hard to handle
- But DQN can achieve superhuman performance on benchmark Atari Games

Source: Otmar Hilliges

- Q-learning is (typically) limited to discrete action spaces
- Learning a policy is often much easier. The algorithm directly learns the correct behaviour without exploring the value function.

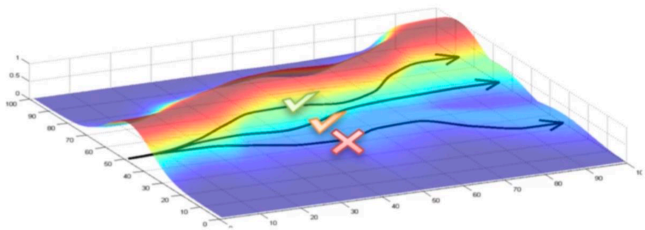
- We evaluate the policy by computing the expectation of the trajectory reward.
- A trajectory τ is a sequence of states and actions $s_1, a_1, \ldots, s_T, a_T$.
- Then we can define a performance meassure

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_t \gamma^t r(s_t, a_t) \right] = \mathbb{E}_{\tau \sim p_\theta(\tau)} [r(\tau)]$$

- Collect rollout trajectories:

$$p(\tau) = p(s_1, a_1, ..., s_T, a_T) =$$

$$p(s_1) \prod_{t=1}^{T} \pi(a_t|s_t) p(s_{t+1}|a_t, s_t)$$



Source: Otmar Hilliges

- Update the policy:
  - Good trajectories made more likely
  - Bad trajectories made less likely

- How should we do updates on the policy parameters?
- The goal is to maximise the performance measure:

$$\theta^* = \arg\max J(\theta)$$

- We update parameters using gradient ascent:

$$\theta = \theta + \nabla_\theta J(\theta)$$

- How can we ensure that an update step through gradient ascent actually improves the policy?
- Show that updates on the gradients of the performance measure can be done without knowledge of the environment dynamics

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[r(\tau)] = \int p_\theta(\tau) r(\tau) d\tau$$

$$\begin{aligned}
\nabla_\theta J(\theta) &= \int \nabla_\theta p_\theta(\tau) r(\tau) d\tau \\
&= \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) r(\tau) d\tau \\
&= \mathbb{E}_{\tau \sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau) r(\tau)]
\end{aligned}$$

$$\text{since } \nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$$

$$
\begin{aligned}
\log p(\tau) &= \log p(s_1, a_1, \ldots, s_T, a_T) \\
&= \log \left[p(s_1) \prod_{t=1}^{T} \pi_\theta(a_t|s_t) p(s_{t+1}|a_t, s_t) \right] \\
&= \log p(s_1) + \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) + \sum_{t=1}^{T} \log p(s_{t+1}|a_t, s_t)
\end{aligned}
$$

Dynamics ($p(s_{t+1} \mid s_t, a_t)$, $p(s_1)$) are independent of θ:

$\Rightarrow \nabla_\theta \log p(s_1) = 0, \quad \nabla_\theta \log p(s_{t+1}|s_t, a_t) = 0$

$\Rightarrow \nabla_\theta \log p_\theta(\tau) = \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)$

$$\begin{aligned}\nabla_\theta J(\theta) &= \mathbb{E}_{\tau\sim p_\theta(\tau)}[\nabla_\theta \log p_\theta(\tau) r(\tau)] \\ &= \mathbb{E}_{\tau\sim p_\theta(\tau)}\left[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) r(\tau)\right]\end{aligned}$$

How can we use these policy updates and take the expected value?

- Sample trajectories τ_i by rolling out the policy, i.e. sampling a action from the current policy until the end of the episode
- Monte Carlo type of RL method - evaluates full trajectories/episodes to update the policy:

$$\nabla_\theta J(\theta) = \frac{1}{N} \sum_i \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \right) \left(\sum_{t=0}^{T} \gamma^t r(s_t^i, a_t^i) \right)$$

- Update the policy parameters:

$$\theta = \theta + \nabla_\theta J(\theta)$$

Monte Carlo sampling produces noisy policy gradients!
Gradients are computed using only few samples

$$\nabla_\theta J(\theta) = \frac{1}{N} \sum_i \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \right) \left(\sum_{t=0}^{T} \gamma^t r(s_t^i, a_t^i) \right)$$

- To reduce the variance we can introduce a baseline $b(s_t^i)$:

$$\nabla J(\theta) = \frac{1}{N} \sum_i \left(\sum_{t=0}^{T} \nabla \log \pi_\theta(a_t^i | s_t^i) \right) \left(\sum_{t=0}^{T} \gamma^t r(s_t^i, a_t^i) - b(s_t^i) \right)$$

- Baseline can be any function that does not depend on the action a_t. Examples:
  - Average reward as the baseline
  - State value function $V(s_t)$ as baseline
- The variance is reduced but the policy gradient estimate stays unbiased.

As an extension of policy gradient algorithm, can we do better?

- To introduce bias and reduce the variance even further, we use bootstrapping for estimating the reward:

$$\nabla_\theta J(\theta) = \frac{1}{N} \sum_i \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i)(r(s_t^i, a_t^i) + \gamma V(s_{t+1}^i) - V(s_t^i))$$

Function approximation instead of whole trajectory rollouts

with $V(s_t)$ as the value function in state s_t, i.e., the expected cumulative reward

- The policy (actor) and the value function (critic) are both represented by neural networks
- Most SOTA approaches build upon the actor-critic concept

$$\nabla_\theta J(\theta) = \frac{1}{N} \sum_i \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i)(r(s_t^i, a_t^i) + \gamma V(s_{t+1}^i) - V(s_t^i))$$

Does this term look familiar?

- It is the TD-error

This term is an estimation of the **advantage function**, which describes the relative advantage of a certain action a over all others:

$$A_t = Q(s_t, a_t) - V(s_t) \approx r(s_t, a_t) + \gamma V(s_{t+1}) - V(s_t) = \hat{A}_t$$

- Trust Region Policy Optimisation (TRPO)
- Proximal Policy Optimisations (PPO)

The following Actor-Critic approaches we leave to the interested reader:

- Deep Deterministic Policy Gradients (DDPG)
- Asynchronous Advantage Actor-Critic (A3C)
- Soft Actor-Critic (SAC)
- TD3

- In supervised learning, the data distribution is fixed
- In reinforcement learning, the policy defines the data distribution
- Updating the policy changes:
  - which actions are taken
  - which states are visited
  - which data is observed next
- A large policy update can completely change the training distribution

$$\text{Small parameter change} \not\Rightarrow \text{small behavior change}$$

- Adjusting step size is much harder compared to supervised learning
  - Small steps lead to slow learning
  - One large step in the wrong direction can have disastrous effects

- Directly optimizing $J(\theta)$ is unstable because policy updates change the data distribution
- TRPO optimizes a **local surrogate objective** around data collected by the old policy $\pi_{\theta_{\text{old}}}$ [Schulman et al., 2015] while updating θ

$$\max_{\theta} \; \mathbb{E}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right]$$

$$\text{s.t.} \quad \mathbb{E}_t \left[D_{KL} \left(\pi_{\theta_{\text{old}}}(\cdot|s_t) \,\|\, \pi_\theta(\cdot|s_t) \right) \right] \leq \delta$$

- Ratio: importance sampling under the old policy
- Constraint: limit how much the policy is allowed to change

- Can be posed in an actor-critic fashion
- The constrained optimisation problem can be solved approximately using the Fisher Information matrix and conjugate gradient method
- It is a second-order optimisation problem and computationally heavy

- Follow up work of TRPO
- Relax the hard constraint condition and turn it into a tunable parameter β

$$\max \mathbb{E}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t \right] - \beta \mathbb{E}_t [D_{KL}(\pi_{\theta_{old}}(\cdot|s_t)||\pi_\theta(\cdot|s_t))]$$

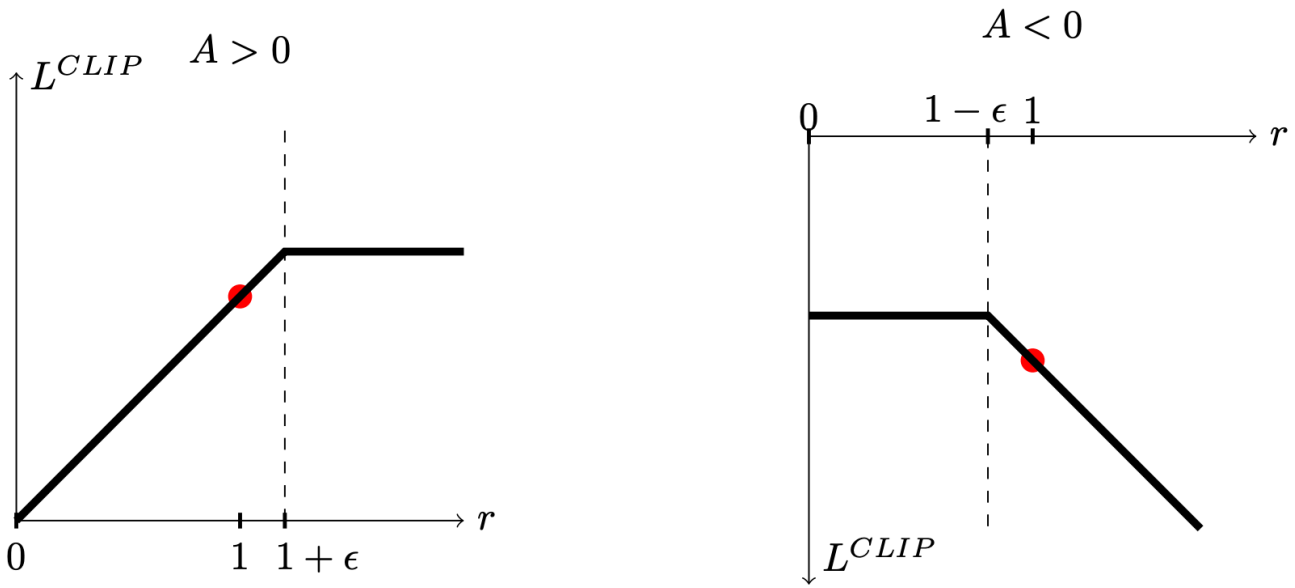
Penalizes deviations from the old policy

- Allowing some bad decisions, but generally optimizing within a trust region [Schulman et al., 2017]

- But we can even do better by using a clipped objective

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t \right) \right]$$

- where the probability ratio is $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$



Source: [Schulman et al., 2017]

We clip the ratio to discourage large policy changes outside of our comfort zone.

- Much simpler implementation than TRPO
- Less computationally expensive (first-order)
- Trade-off between fast optimisation and allowing some bad updates

Algorithm 5 PPO with Clipped Objective

Input: initial policy parameters θ_0, clipping threshold ϵ
for $k = 0, 1, 2, \ldots$ **do**
  Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$
  Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm
  Compute policy update

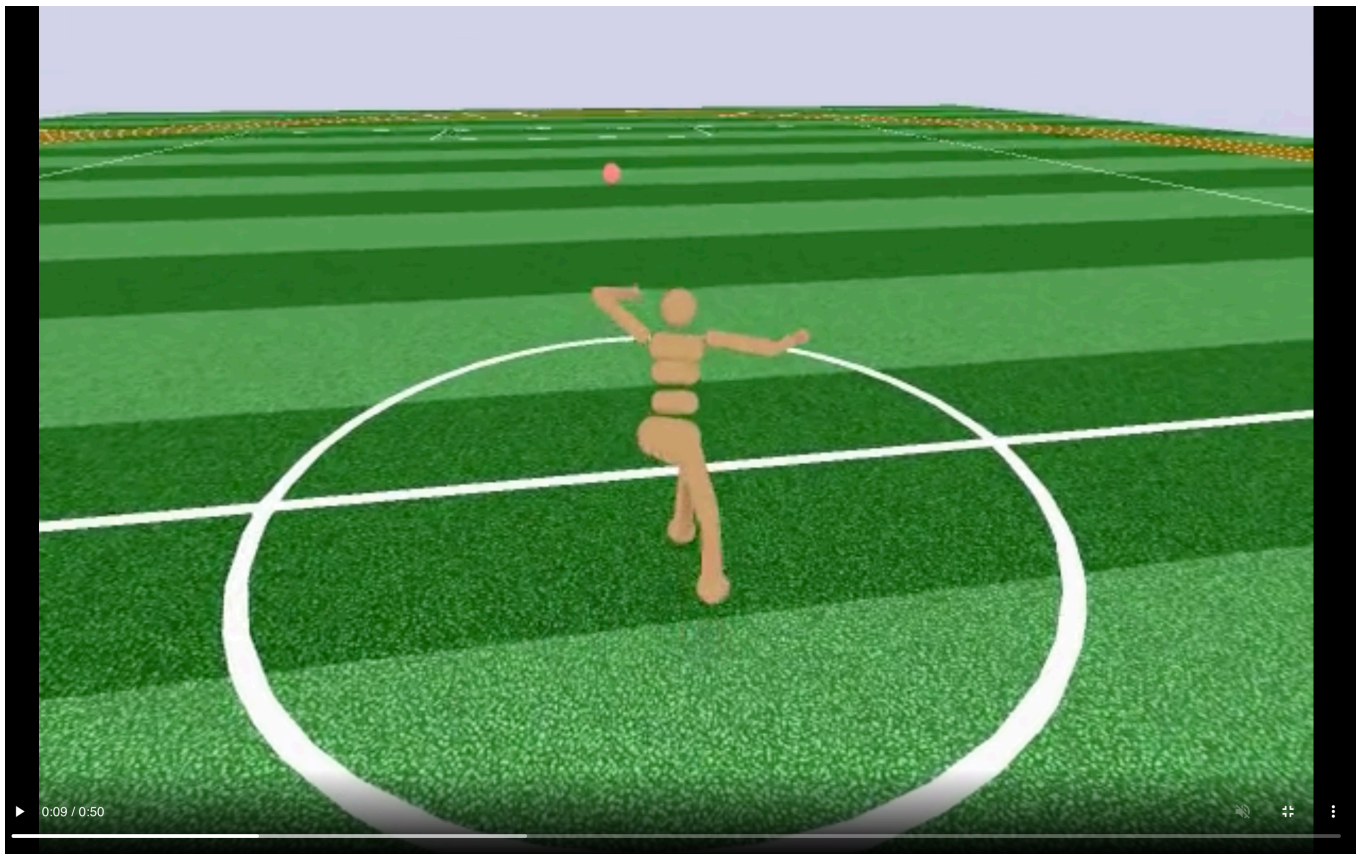
$$\theta_{k+1} = \arg\max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

  by taking K steps of minibatch SGD (via Adam), where

$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathop{\mathrm{E}}_{\tau \sim \pi_k} \left[\sum_{t=0}^{T} \left[\min(r_t(\theta)\hat{A}_t^{\pi_k}, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t^{\pi_k}) \right] \right]$$

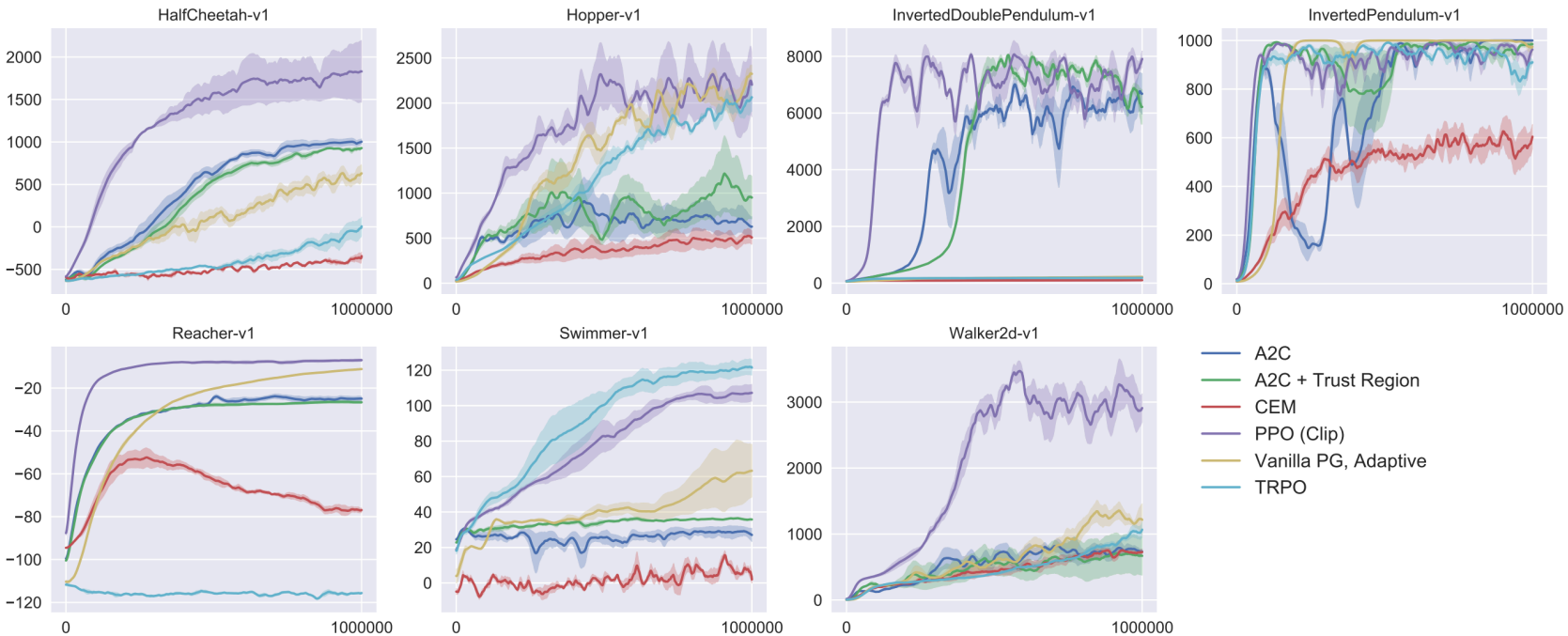
end for

Source: Source: Schulman [2017]



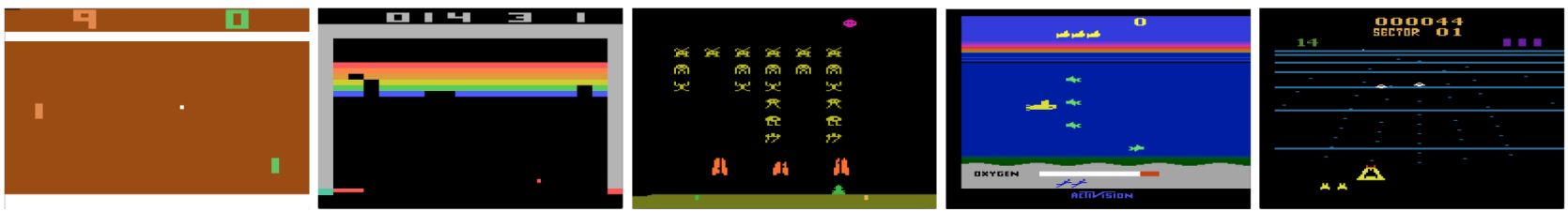
Source: Source: https://openai.com/blog/openai-baselines-ppo/

Deep Reinforcement Learning – PPO vs. TRPO vs. Actor-Critic vs. Policy Gradients



Source: [Schulman et al., 2017]

Deep Reinforcement Learning – Example Achievement: Atari



Source: Mnih et al. [2013]

Deep Reinforcement Learning – Example Achievement: Alpha Go

Source: Silver et al. [2016]

Source: Schulman et al. [2017]

Deep Reinforcement Learning – Example Achievement: StarCraft

Source: Vinyals et al. [2019]

S. V. Albrecht, F. Christianos, and L. Schäfer. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. URL *https://www.marl-book.com*.

V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*, 2013.

J. Schulman. Deep Reinforcement Learning via Policy Optimization, tutorial, 2017.

J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust Region Policy Optimization. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.